

Introduction to Algorithm



Dr. Sanjay Kumar
Assistant Professor,
Dept. of CSE,
Delhi Technological University

Reference Book

- Introduction to Algorithms by CLRS
- Data Structures and Algorithms by Aho, Hopcroft and Ullman
- Fundamentals of Computer Algorithm by Horowitz, Sahni and Rajasekaran

Algorithm

Basic goals for an algorithm:

- always correct
- always terminates
- takes input
- Produces output
- performance

Algorithm

It is a finite set of instructions written in any user friendly language to solve a particular problem.

Design and Analysis of Algorithms

- ***Design:*** design algorithms which minimize the cost
- ***Analysis:*** predict the cost of an algorithm in terms of resources and performance

Analysis of algorithm

- Calculating or computing all the resources that algorithm requires to solve the problem
- Name some resources:-
 - main memory size
 - time taken
 - no of processor,
 - clock speed
 - network speed

hard disk space

Time for input/output

OS

Hardware requirement

Space requirement

Analysis of algorithm

Calculating or computing the time and space requires to solve the problem **in terms of input size**

Factor affecting time complexity of an algorithm

- **Data and data structure type**
- **Constructs used**
- **PL**
- **OS**
- **Processor Speed**
- **Hardware configuration**
- **Input size**

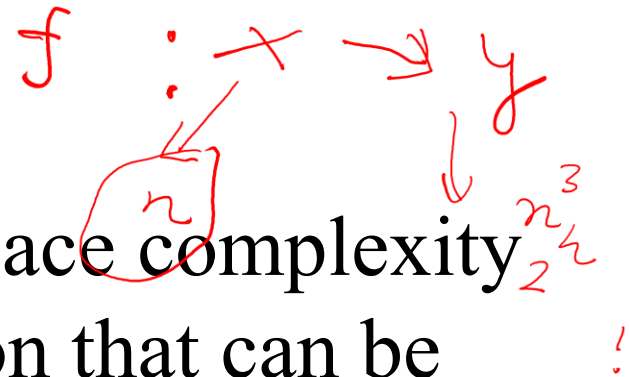
Time complexity of Algorithm

- It is the total time required by an algorithm to complete the execution in **terms of input size**.

Space complexity of Algorithm

- It is the total memory or space required by an algorithm to complete the execution in terms of input size.

- Basically, both time and space complexity of an algorithm is a function that can be expressed in terms of input size.



The RAM Computer Model

- The random-access-machine(RAM)
 - Single processor
 - Unit time addressable memory
 - Unit time multiplication and addition of numbers
 - Each instruction takes constant time and it is different for other instruction.

Analysis of Algorithm

- Analyzing Time and space complexity
- Basically, both time and space complexity of an algorithm is a function that can be expressed in **terms of input size**.

Machine-independent time

Main IDEA:

- *Ignore machine dependent constants*,
otherwise impossible to verify and to compare algorithms
- Look at *growth* of $T(n)$ as $n \rightarrow \infty$.

“Asymptotic Analysis”

Kinds of asymptotic analyses

Worst-case: (usually)

$T(n)$ = maximum time of algorithm on any input of size n .

- Default case

Average-case: (sometimes)

- $T(n)$ = expected time of algorithm over all inputs of size n .
- Need assumption of statistical distribution of inputs.

Best-case: (NEVER)

- Case when an algorithm works fast on *some* input
- Special case
- Don't rely on best case

If $T(n) = O(1)$, implies algorithm runs in constant time irrespective of input size, n

If $T(n) = O(n)$, implies performance of algorithm grows linearly and is directly proportional to input size, n

Analysis of Algorithm

It is a comparative analysis. Lets A and B are two algorithms.

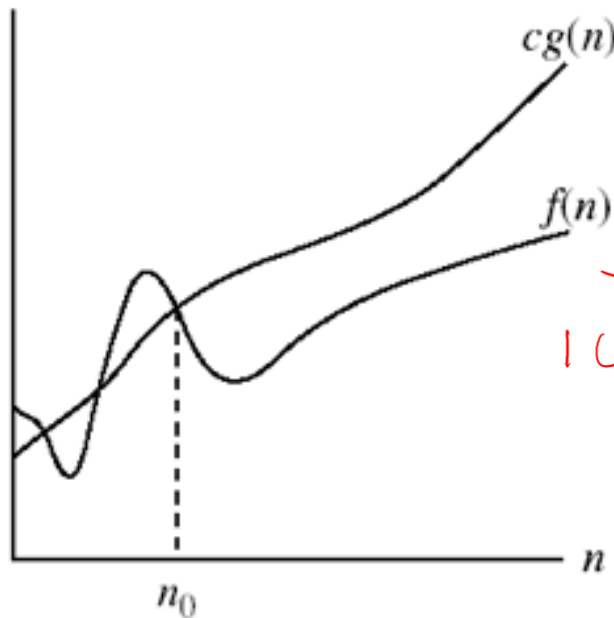
1. $A = O(B)$, A takes less time then B

2. $A = \Theta(B)$, A and B take similar time

3. $A = \Omega(B)$, A takes more time then B

Worst case analysis O -notation

$f(n) = O(g(n))$ if and only if there are positive constants c and n_0 such that $f(n) \leq c g(n)$ for $n > n_0$.



n	2^n	n^3
1	2	1
2	4	8
3	8	27
4	16	64
5	32	125
10	1024	1000

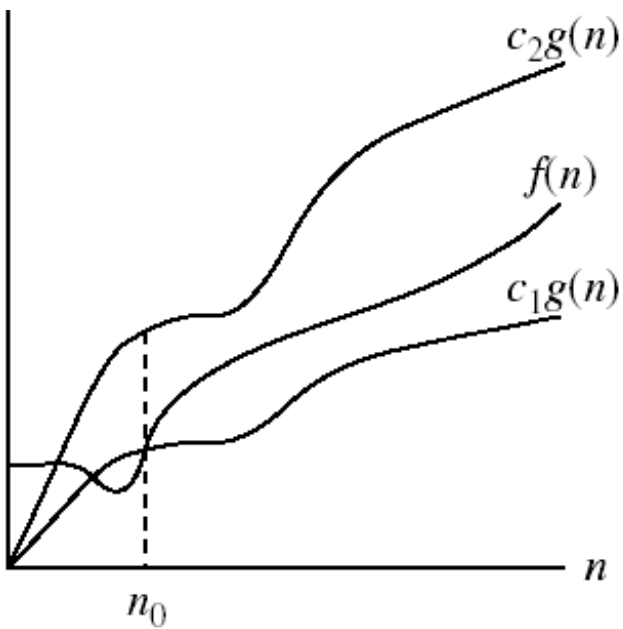
$$\forall n > 10, 2^n > n^3$$

$g(n)$ is an asymptotic upper bound for $f(n)$. $n^3 = O(2^n)$

Θ -notation

$f(n) = \Theta(g(n))$, implies there are positive constants c_1, c_2 and n_0 such that

$$c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for } n \geq n_0$$

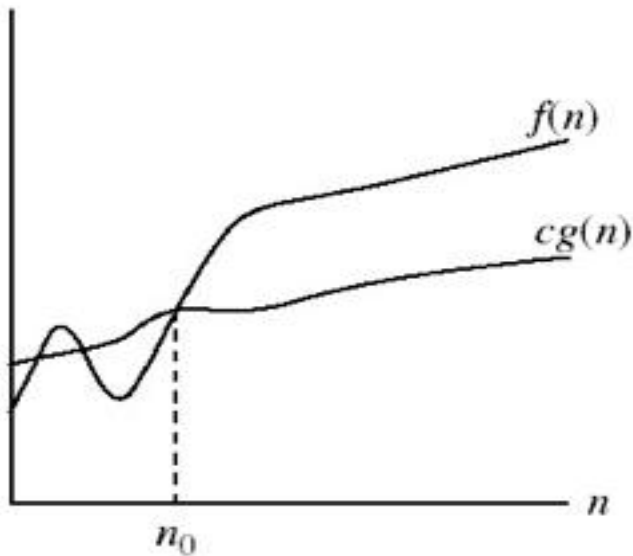


$f(n)$ is the set of functions
with the same order of growth
as $g(n)$

$g(n)$ is an *asymptotically tight bound* for $f(n)$.

Best case Analysis Ω - notation

$f(n) = \Omega(g(n))$ if and only if there are positive constants c and n_0 such that $c g(n) \leq f(n)$ for all $n \geq n_0$



$g(n)$ is an *asymptotic lower bound* for $f(n)$.

If $f(n) = 2^n$ and $g(n) = n^3$

Then which one is true

$F(n) = O(g(n))$ Or $g(n) = O(f(n))$

Comparison of growth of various functions

- $O(1) < O(\log \log n) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!) < O(n^n)$

Time Complexity of non-recursive Algorithm

Computing Time complexity of a non-recursive Algorithm

Find the frequency count of each statement and total time is sum of product of time of each instruction and its frequency.

We should adapt RAM model.

We ignore the constant factors

Computing Time complexity of non-recursive Algorithm

Assumption:

Time required for each instruction is constant and is different for different instruction.

Time complexity of a non-recursive program or algorithm is nothing but frequency of the statement which is executing for the maximum time

- Find the time complexity of the following code Q1.

```
int i, j, k = 0;
for (i = 0; i < n; i++)
    for (j = 0; j < n; j++)
        k++;
```

Handwritten analysis of the code's time complexity:

The code consists of four statements, each with an associated time t_i and frequency f_i :

- Statement 1: t_1 (initialization of i, j, k), frequency 1.
- Statement 2: t_2 (loop condition $i < n$), frequency $n+1$.
- Statement 3: t_3 (loop condition $j < n$), frequency $n(n+1)$.
- Statement 4: t_4 (body $k++$), frequency $n \times n$.

The total time T is calculated as:

$$T = \sum_{i=1}^n t_i \times f_i$$

$$= t_1 + t_2(n+1) + t_3n(n+1) + t_4n^2$$

$$= (t_1 + t_2) + n(t_2 + t_3) + n^2(t_3 + t_4)$$

$$= c_1 + n c_2 + n^2 c_3$$

$$= O(n^2)$$

Find the time complexity of the following code
Q2.

```
int i, j, k , p=0;  
for (i = 0; i < n; i++)  
    for (j = 0; j < n; j ++)  
        for (k  = 0; k < n; k  ++)  
            p++;
```

Find the time complexity of the following code
Q3.

```
int i, j, k , p=0;  
for (i = 0; i < n; i++)  
    for (j = 0; j < n; j ++)  
        for (k = 0; k <=1000; k ++)  
            p++;
```

- Find the time complexity of the following codes

Q1.

```
while(n>1)
    n=n/2;
```

Q2.

```
while(n>1)
    n=n/3;
```

Q3.

```
while(n>1)
    n=n/3;
    n=n/15;
```

Computing Time complexity of recursive Algorithm

For non-recursive Algorithm-

Find the frequency count of each statement and total time is sum of product of time of each instruction and its frequency.

We should adapt RAM model.

Find the time complexity of the following function

F(n)

```
{  
  int i, j, k = 0;  
  for (i = n / 2; i < n; i++) {  
    for (j = 2; j < n; j = j * 2) {  
      k = k + n / 2;  
    }  
  }  
}
```

O(n)

O(nLogn)

O(n²)

O(n²Logn)

What is time complexity of foo()?

```
int foo(int n)
{
    int count = 0;
    for (int i = n; i > 0; i /= 2)
        for (int j = 0; j < i; j++)
            count += 1;
    return count;
}
```

(A) $O(n^2)$

(B) $O(n \log n)$

(C) $O(n)$

(D) $O(n \log n \log n)$

What is time complexity of foo()?

```
int foo(int n)
{
    int count = 0;
    for (int i = n; i > 0; i /= 2)
        for (int j = 0; j < i; j++)
            count += 1;
    return count;
}
```

- (A) $O(n^2)$
- (B) $O(n \log n)$
- (C) $O(n)$
- (D) $O(n \log \log n)$

$$\begin{aligned} & - \log n \\ & n + \frac{n}{2} + \frac{n}{4} + \dots \rightarrow \log n \text{ terms} \\ & = n \left(1 + \frac{1}{2} + \frac{1}{4} + \dots \right) \\ & \quad \log_2 n \text{ terms} \\ & = n \left[1 \cdot \left(1 - \left(\frac{1}{2} \right)^n \right) \right] \end{aligned}$$

Q Find the time complexity of the given function

```
int fun(int n)
{
    int count = 0;
    for (int i = 0; i < n; i++)
        for (int j = i; j > 0; j--)
            count = count + 1;
    return count;
}
```

- (A)** $O(n^2)$
- (B)** $O(n \log n)$
- (C)** $O(n)$
- (D)** $O(n \log n \log n)$

Q1 Consider the following three claims

1. $(n + k)^m = \Theta(n^m)$, where k and m are constants
2. $2^{n+1} = O(2^n)$
3. $2^{2n+1} = O(2^n)$

Which of these claims are correct ?

- (A) 1 and 2
- (B) 1 and 3
- (C) 2 and 3
- (D) 1, 2, and 3

Q2. Which of the given options provides the increasing order of asymptotic complexity of functions f_1 , f_2 , f_3 and f_4 ?

$$f_1(n) = 2^n$$

$$f_2(n) = n^{(3/2)}$$

$$f_3(n) = n \log n$$

$$f_4(n) = n^{(\log n)}$$

(A) f_3, f_2, f_4, f_1

(B) f_3, f_2, f_1, f_4

(C) f_2, f_3, f_1, f_4

(D) f_2, f_3, f_4, f_1

Time Complexity of recursive Algorithm

Time Complexity

Computing time complexity of a recursive Algorithm-

- Find the suitable recurrence relation for the algorithm
- Solve the recurrence using methods like- iteration, substitution, recursion-tree, master theorem.
- Solution of the recurrence relation will be the time complexity of the algorithm

Definition

- A recurrence relation, $T(n)$, is a recursive function of integer variable n .
- Like all recursive functions, it has both recursive case and base case.
- Example:

$$T(n) = \begin{cases} a & \text{if } n = 1 \\ 2T(n/2) + bn + c & \text{if } n > 1 \end{cases}$$

- The portion of the definition that does not contain T is called the **base case** of the recurrence relation; the portion that contains T is called the **recurrent or recursive case**.
- Recurrence relations are useful for expressing the running times (i.e., the number of basic operations executed) of recursive algorithms

- Find the recurrence relation

```
int factorial (int n) {  
    if (n == 0)    return 1;  
    else  
        return n * factorial (n - 1);  
}
```

```
void f(int n) { if (n
    > 0) {
cout<<n;  f(n-1);
} }
```

- The base case is reached when $n == 0$. The method performs one comparison. Thus, the number of operations when $n == 0$, $T(0)$, is some constant a .
- When $n > 0$, the method performs two basic operations and then calls itself, using ONE recursive call, with a parameter $n - 1$.
- Therefore the recurrence relation is:

$$T(0) = a$$

$$T(n) = b + T(n-1)$$

where a is constant

where b is constant, $n > 0$

Eg-2

```
int g(int n) {  
  if (n == 1) return 2;  
  else  
    return 3 * g(n / 2) + g(n / 2) + 5;  
}
```

$T(n)$

$T(n/2)$

$T(n/2)$

$$T(10) = 3 \times T(5) + T(5) + 5$$

Forming recurrence relation

- Example 2: Write the recurrence relation for the following method.

```
int g(int n) {  
    if (n == 1)    return 2;  
    else  
        return 3 * g(n / 2) + g(n / 2) + 5;  
}
```

- The base case is reached when $n == 1$. The method performs one comparison and one return statement. Therefore, $T(1)$, is constant c .
- When $n > 1$, the method performs TWO recursive calls, each with the parameter $n/2$, and some constant # of basic operations.
- Hence, the recurrence relation is:

$$T(1) = c$$

for some constant c

$$T(n) = b + 2T(n/2)$$

for a constant b

Eg-3

```
Int f(int n) {  
  if (n == 1)  return 2;  
  else  
  Return n * f(n / 2) * f( n / 2) + n2;  
}
```

Analysis Of Recursive Binary Search

```
int BS (int target, int[] a,int low, int high)
{
if (low > high)    return -1;
else {
int mid = (low + high)/2;
if (a[mid] == target)
return mid;
else if(a[mid] < target)
return BS(target, a, mid + 1, high);   else
return BS(target, a, low, mid- 1);
}
}
```

- The recurrence relation for the running time of the method is:

$$T(1) = a \quad \text{if } n = 1 \quad (\text{one element array})$$

$$T(n) = T(n / 2) + b \quad \text{if } n > 1$$

Example: Fibonacci Sequence

- The Fibonacci sequence is defined by the recurrence relation
 - $f_n = f_{n-1} + f_{n-2}$, $n \geq 3$ and initial conditions
 - $f_1 = 1, f_2 = 1$.
- Fibonacci Sequence: 1, 1, 2, 3, 5, 8, 13, 21,.....

Solving Recurrence Relation

- To solve a recurrence relation $T(n)$ we need to derive a form of $T(n)$ that is not a recurrence relation. Such a form is called a “*closed form*” of the recurrence relation.
- There are four methods to solve recurrence relations that represent the running time of recursive methods:
 - Iteration method (*unrolling and summing*)
 - Recursion Tree method
 - Substitution method
 - Master method

Iteration method

● Steps:

- Expand the recurrence
- Express the expansion as a summation by plugging the recurrence back into itself until you see a pattern.
- Evaluate the summation

Solving Recurrence

Relations - Iteration method

- In evaluating the summation one or more of the following summation formulae may be used:
- Arithmetic series:

$$\sum_{k=1}^n k = 1 + 2 + \dots + n = \frac{n(n+1)}{2}$$

- Geometric Series:

$$\sum_{k=0}^n x^k = 1 + x + x^2 + \dots + x^n = \frac{x^{n+1} - 1}{x - 1} (x \neq 1)$$

$$\sum_{k=0}^{n-1} x^k = \frac{x^n - 1}{x - 1} (x \neq 1)$$

- Special Cases of Geometric Series:

$$\sum_{k=0}^{n-1} 2^k = 2^n - 1$$

$$\sum_{k=0}^{\infty} x^k = \frac{1}{1-x} \quad \text{if } |x| < 1$$

Solving Recurrence

Relations - Iteration method

- Harmonic Series:

$$\sum_{k=1}^n \frac{1}{k} = 1 + \frac{1}{2} + \dots + \frac{1}{n} \approx \ln n$$

- Others:

$$\sum_{k=1}^n \lg k \approx n \lg n$$

$$\sum_{k=0}^{n-1} c = cn.$$

$$\sum_{k=0}^{n-1} \frac{1}{2^k} = 2 - \frac{1}{2^{n-1}}$$

$$\sum_{k=1}^n k^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{k=0}^n k(k+1) = \frac{n(n+1)(n+2)}{3}$$

Analysis Of Recursive Factorial

Method

- Example 1: Form and solve the recurrence relation for the running time of factorial method and hence determine its big-O complexity:

```
long factorial (int n) {  
    if (n == 0)    return 1;  
    else  
        return n * factorial (n - 1);  
}
```

$$T(o) = c$$

$$T(n) = b + T(n - 1)$$

$$= b + b + T(n - 2)$$

$$= b + b + b + T(n - 3)$$

...

$$= kb + T(n - k) \text{ When } k$$

= n, we have:

$$T(n) = nb + T(n - n)$$

$$= bn + T(o)$$

$$= bn + c.$$

Therefore method factorial is $O(n)$.

Analysis Of Recursive Binary Search

Expanding:

$$\begin{aligned}T(n) &= T(n/2) + b \\&= [T(n/4) + b] + b = T(n/2^2) + 2b \\&= [T(n/8) + b] + 2b = T(n/2^3) + 3b \\&= \dots\dots\dots \\&= T(n/2^k) + kb\end{aligned}$$

When $n/2^k = 1 \rightarrow n = 2^k \rightarrow k = \log_2 n$, we have: $T(n) = T(1)$

$$\begin{aligned}&+ b \log_2 n \\&= a + b \log_2 n\end{aligned}$$

Therefore, Recursive Binary Search is $O(\log n)$

Recursion-tree method

- A recursion tree models the costs (time) of a recursive execution of an algorithm.
- The recursion tree method is good for generating guesses for the substitution method.
- The recursion-tree method can be unreliable.
- The recursion-tree method promotes intuition, however.
- In this case, only the largest term in the geometric series matters; all of the other terms are swallowed up by the $\Theta(\cdot)$ notation.

- The **recursion tree method** is a way of visualizing how a recurrence relation expands by drawing a tree where:
- Each node represents the cost of a subproblem.
- Children represent the subproblems generated by recursion.
- The total work is the sum of the costs across all levels of the tree.

◆ General Steps

Suppose we have a recurrence of the form:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

where:

- a = number of subproblems
- n/b = size of each subproblem
- $f(n)$ = cost outside the recursive calls (like splitting, merging, etc.)

Steps:

1. Draw the root node with cost $f(n)$.
2. Expand into children: Each child represents a subproblem of size n/b , each costing $f(n/b)$. Since there are a subproblems, there are a children.
3. Keep expanding until the subproblem size becomes 1 (base case).
4. Add up the costs across all levels of the tree.
5. Compute the total cost = sum of costs from all levels.

◆ Example 1: Merge Sort

Recurrence:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

Step 1: Root

- Cost at root = $O(n)$.

Step 2: First level

- Two subproblems of size $n/2$.
- Each has cost $O(n/2)$.
- Total at this level = $2 \cdot O(n/2) = O(n)$.

Step 3: Second level

- Four subproblems of size $n/4$.
- Each has cost $O(n/4)$.
- Total = $4 \cdot O(n/4) = O(n)$.

Step 4: Keep going

At each level, the total cost = $O(n)$.

There are $\log_2 n$ levels (because problem size halves each time).

Step 5: Total cost

$$T(n) = O(n \log n)$$

Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:

↑
Root

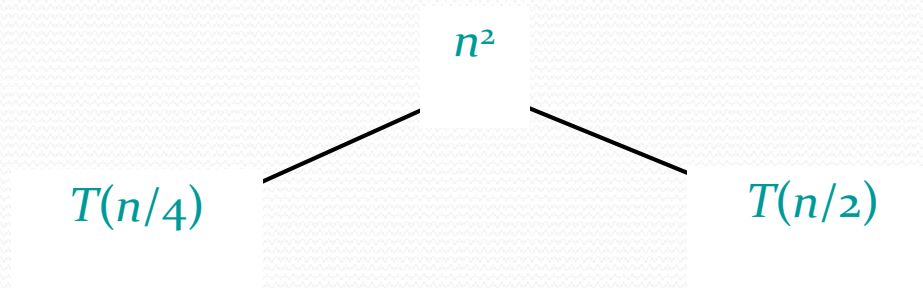
Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:

$$T(n)$$

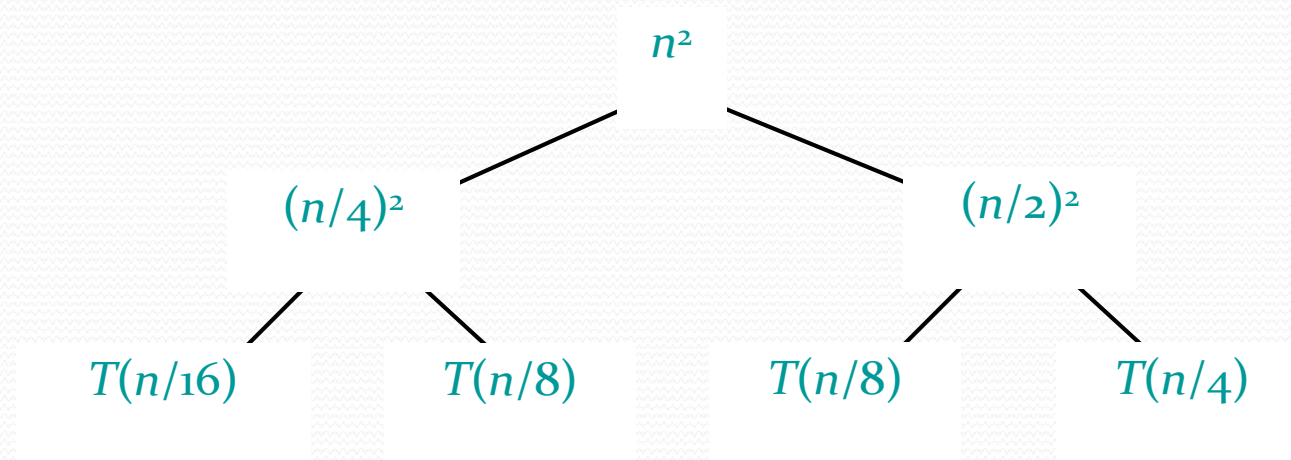
Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:



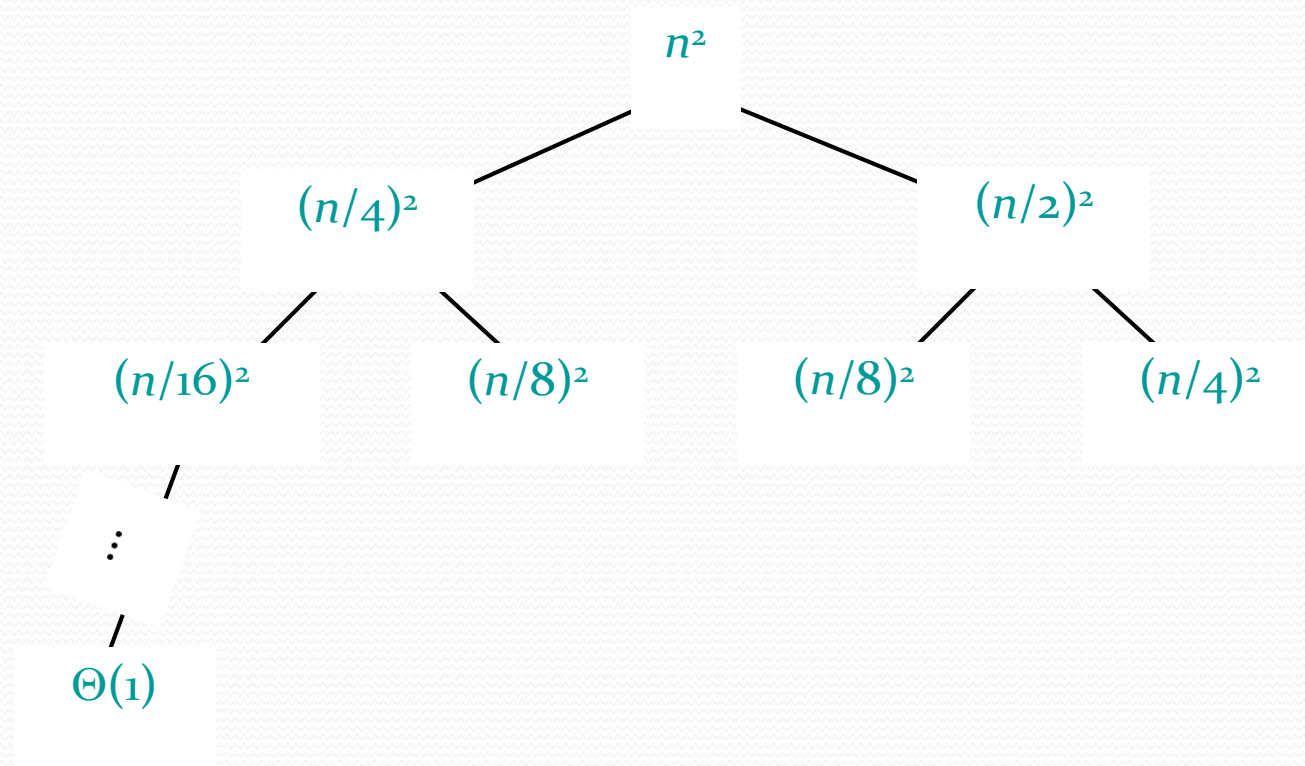
Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:



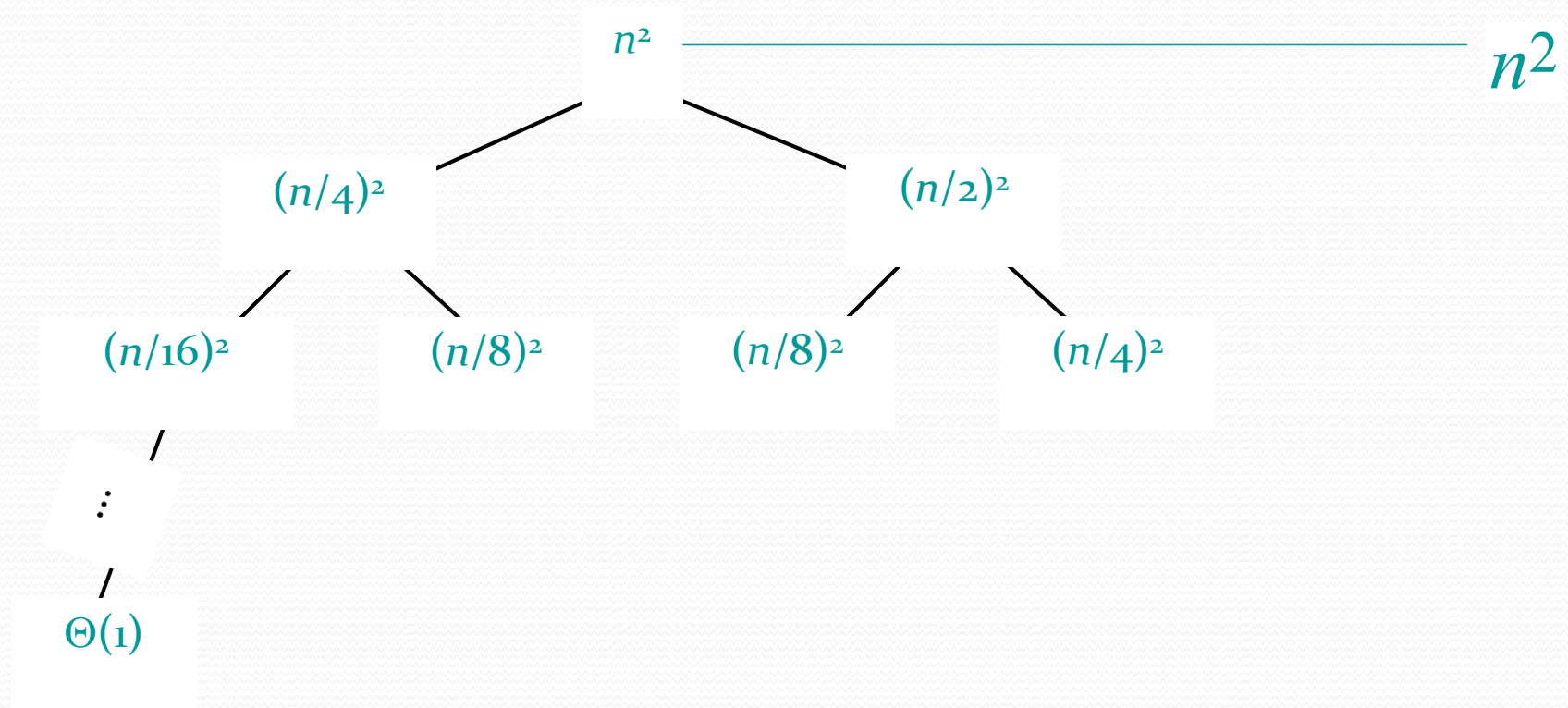
Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:



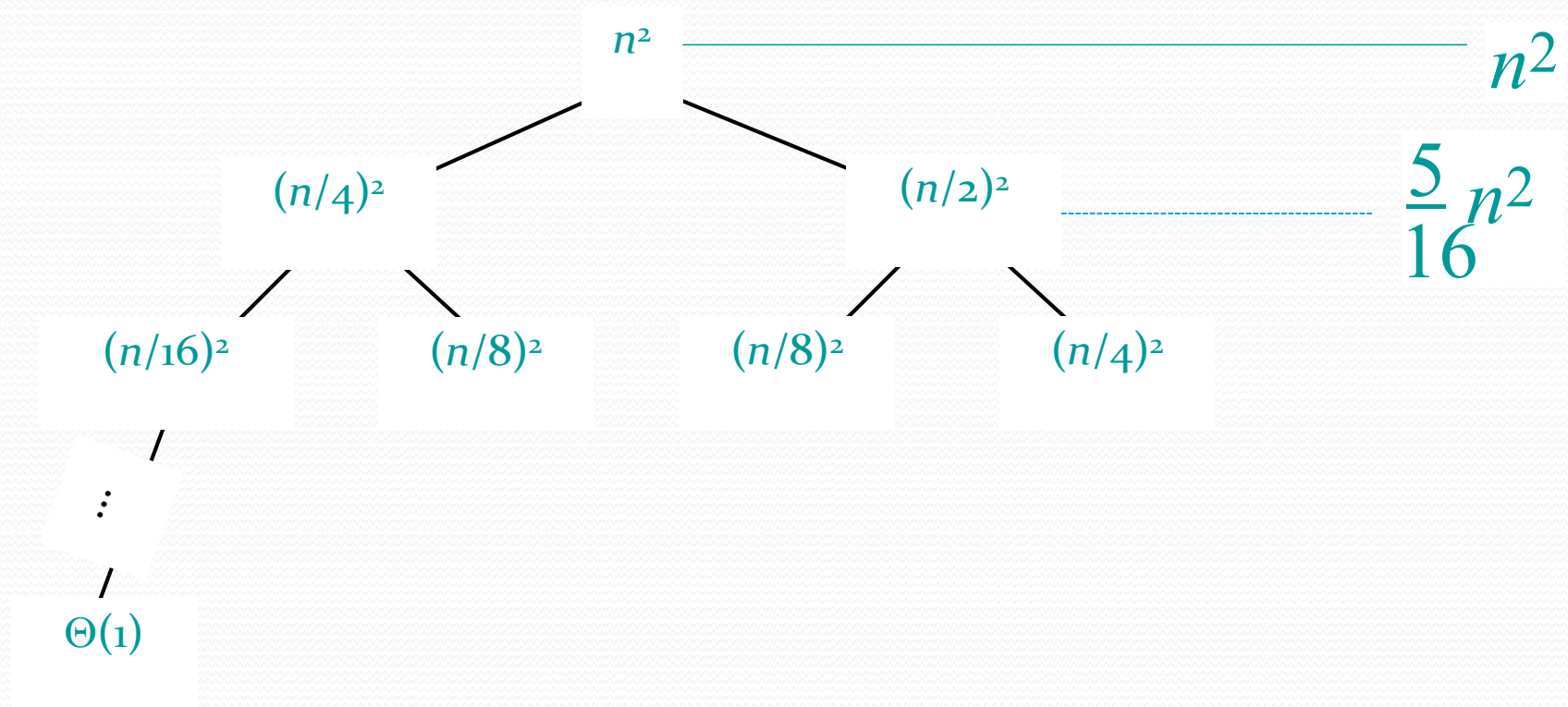
Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:



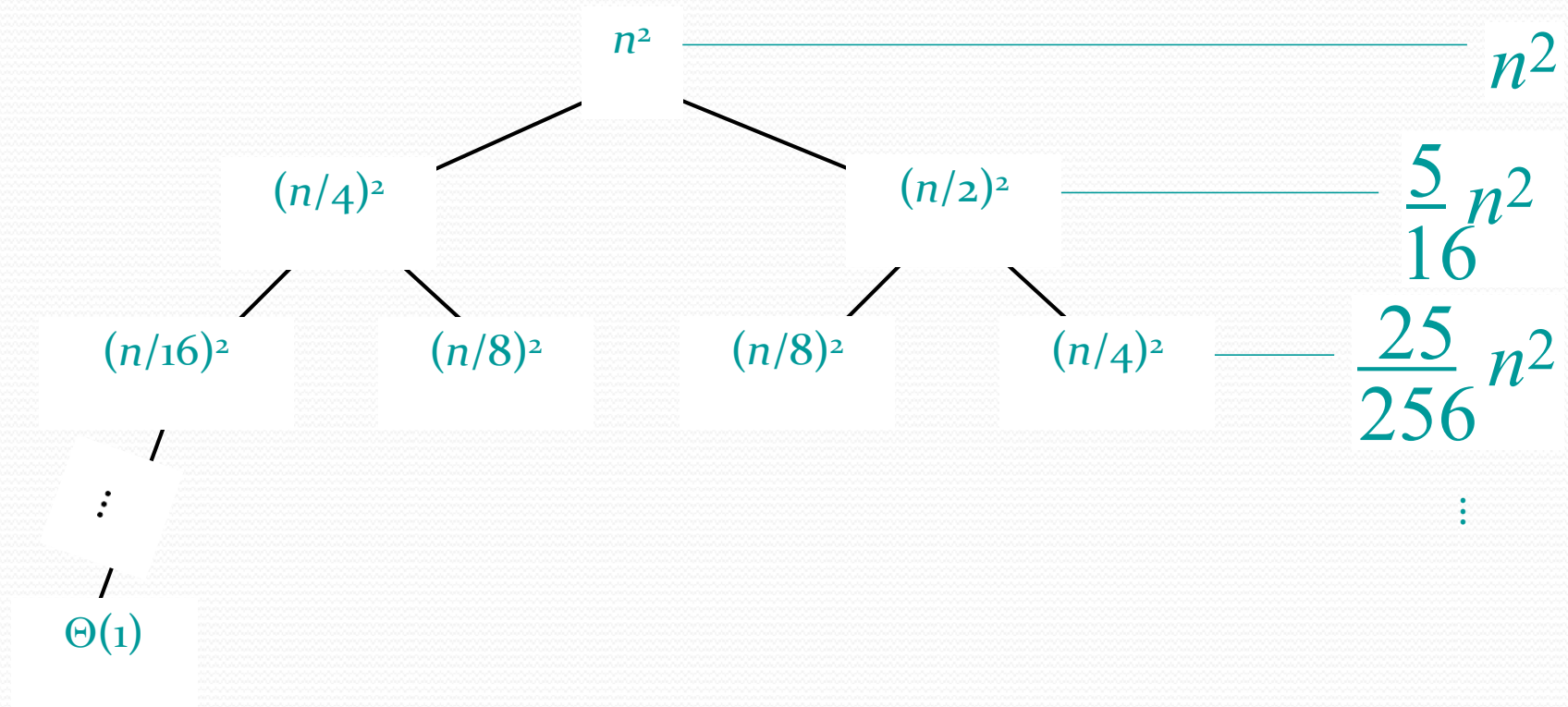
Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:



Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:



Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:

